

GUÍA TÉCNICA DE CONTENEDORIZACIÓN

Docker & Docker Compose para E-Commerce Gamer Fullstack

Trabajo Integrador Final | UTN FR Avellaneda | Tecnicatura en Programación

Autor: Santiago Ezequiel Chavez | Versión: 1.2.0 (Septiembre 2026)

1 ¿Qué es Docker y por qué lo implementamos?

Docker es una plataforma de virtualización a nivel de sistema operativo que permite empaquetar una aplicación junto con todas sus dependencias (código, runtime de Java, Node, servidor Nginx, librerías y configuraciones) dentro de una unidad estandarizada y aislada llamada Contenedor.

El Problema del "En mi máquina sí funciona"

Tradicionalmente, para correr este proyecto un evaluador debía instalar Java 21, Maven, Node.js, configurar variables de entorno en Windows y lidiar con conflictos de versiones. Con Docker, la aplicación corre de forma idéntica en cualquier máquina (Windows, Linux, macOS) garantizando Cero Fricción y reproducible al 100%.

Conceptos Fundamentales que debes dominar:

Concepto	Definición	Analogía en Programación
Dockerfile	Archivo de texto con instrucciones paso a paso para construir una imagen.	Es como una Clase (Molde o Blueprint).
Imagen Docker	Paquete inmutable y autocontenido con el código, runtime y librerías.	Es como el binario compilado (.exe / .jar).
Contenedor	Instancia viva en ejecución y aislada de una imagen Docker.	Es como un Objeto instanciado en memoria.
Docker Compose	Herramienta para orquestar y coordinar múltiples contenedores interconectados.	El archivo "main" que levanta backend y frontend juntos.

2 Arquitectura de Contenedores del E-Commerce

El ecosistema está orquestado mediante 2 servicios en contenedores independientes que interactúan con la base de datos en la nube:

Servicio	Tecnología	Puerto Interno	Puerto Expuesto	Propósito
ecommerce-backend	Spring Boot 3 + JRE 21 Alpine	8080	8080	API REST, persistencia NoSQL, validación de stock
ecommerce-frontend	React 18 + Vite + Nginx Alpine	80	5176	SPA, catálogo gamer, routing de vistas, carrito y PDF
MongoDB Atlas	Cluster Cloud NoSQL	27017	En la nube	Persistencia permanente y distribuida de datos

3 Desglose del Dockerfile del Backend (Multi-Stage Build)

Para que el contenedor sea liviano y seguro, utilizamos el patrón Multi-Stage Build (Construcción en Múltiples Etapas):

BACKEND/DOCKERFILE

```
# ETAPA 1: COMPILACIÓN CON MAVEN (Descarta después)
FROM maven:3.9.6-eclipse-temurin-21-alpine AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B # Guarda dependencias en caché
COPY src ./src
RUN mvn clean package -DskipTests # Genera el archivo .jar

# ETAPA 2: IMAGEN FINAL LIGERA (SOLO RUNTIME JRE 21)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser # Usuario sin privilegios por seguridad
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

¿Por qué es fundamental el Multi-Stage Build?

Maven y el JDK completo superan los 600MB. Con Multi-Stage Build, el contenedor final SOLO contiene el JRE ligero (menos de 150MB) y el JAR compilado. El código fuente original y el compilador Maven se descartan, reduciendo drásticamente la superficie de ataque y el uso de memoria.

4 Desglose del Dockerfile del Frontend y Nginx

En producción no usamos el servidor de desarrollo de Vite. En su lugar, generamos los archivos estáticos HTML/JS/CSS y los servimos con Nginx:

FRONTEND/DOCKERFILE

```
# ETAPA 1: BUILD ESTÁTICO DE REACT
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build # Genera la carpeta /dist optimizada

# ETAPA 2: SERVIDOR NGINX ALPINE ULTRA LIVIANO
FROM nginx:alpine
WORKDIR /usr/share/nginx/html
RUN rm -rf .//*
COPY --from=build /app/dist .
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

¿Por qué necesitamos frontend/nginx.conf? (El problema del SPA Routing)

En las aplicaciones SPA de React Router, al recargar en rutas como "/gestion" o "/pedidos", el navegador le pide al servidor ese archivo físico. Si Nginx no está configurado, devolvería un error 404. La directiva `try_files` redirige todas las rutas a `index.html` para que React maneje la navegación:

FRONTEND/NGINX.CONF

```
server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html; # Soluciona 404 en rutas React
    }
}
```

5 Orquestación con docker-compose.yml

Docker Compose permite coordinar ambos contenedores, configurar puertos, variables de entorno y orden de inicio con un único comando:

DOCKER-COMPOSE.YML

```
services:
  backend:
    build: ./backend
    container_name: ecommerce-gamer-backend
    ports:
      - "8080:8080" # [Puerto en tu PC]:[Puerto en el contenedor]
    environment:
      - SPRING_DATA_MONGODB_URI=mongodb+srv://... (MongoDB Atlas)
      - CORS_ALLOWED_ORIGINS=http://localhost:5176,http://localhost:80
    restart: unless-stopped

  frontend:
    build: ./frontend
    container_name: ecommerce-gamer-frontend
    ports:
      - "5176:80" # Mapea el puerto 80 de Nginx al 5176
    depends_on:
      - backend # Garantiza que el backend inicie primero
    restart: unless-stopped
```

6 Guía Rápida de Comandos (Docker Cheatsheet)

Comando	Descripción
docker compose up --build	Compila las imágenes y levanta todos los contenedores mostrando logs.
docker compose up -d	Levanta los contenedores en segundo plano (modo detached).
docker compose ps	Muestra el estado de salud y puertos de los contenedores.
docker compose logs -f	Sigue los logs en tiempo real de todos los servicios.
docker compose down	Detiene y destruye los contenedores de forma limpia.
docker images	Lista todas las imágenes descargadas y construidas en el equipo.
docker exec -it <nombre> sh	Abre una consola dentro del contenedor en ejecución.

“ Valor Académico para la Evaluación Final

La contenedorización demuestra dominio en DevOps, microservicios, seguridad no-root, optimización de imágenes (Multi-stage) y servidores de producción (Nginx). Esto ubica el proyecto en un nivel profesional y competitivo.